



WELCOME





C++ Functions

Agenda

C++ Bridge Course

Functions

Pass by
value/reference

Practice
Questions

Functions



v/s



Introduction

A function is a *predefined set of instructions* that takes *input*, performs a *specific task*, and returns an *output*, enabling an efficient way to *reuse code* for *repetitive tasks*.

Syntax

```
return_type function_name(data_type parameter)
{
    // Function code or instructions
    return result;
}
```

Example

C++ 

```
#include <iostream>
using namespace std;
int sum(int a, int b){
    int c;
    c = a+b;
    return c;
}
```

Example

C++ 

```
int main() {  
    int a,b;  
    cin >> a >> b;  
    cout << sum(a,b);  
    return 0;  
}
```

Input 

5
10

Output 

15

Comparison with Python:

C++ 

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int main() {
    cout << add(5, 3);
    return 0;
}
```

Output 

8

Python 

```
def add(a, b):
    return a + b

print(add(5, 3))
```

Output 

8

Function Declaration (Prototype)

C++



```
int add(int, int); // Function prototype

int main() {
    cout << add(5, 3);
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

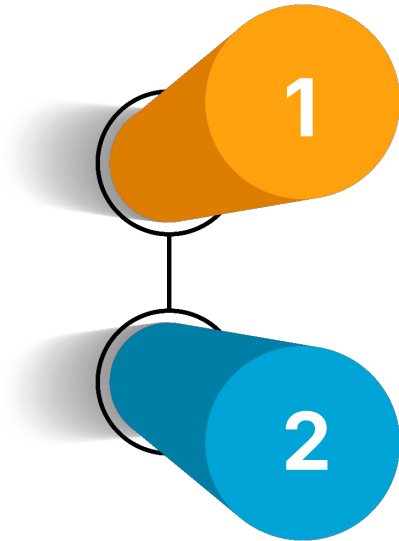
In C++, a **function** must be **declared before use** to **specify its return type and parameters**, allowing calls before its definition.

Python



Python **lacks** function declarations, so a function **must be defined before use**.

Types of Functions



Built-in Functions

User-defined Functions

Types of Functions

Built-in Functions

C++



```
sqrt(), pow(), abs(), max(), min()
```

*These are few of the predefined functions provided by the **C++ Standard Library**.*

Python



```
abs(), pow(), sorted(), sum()
```

*Python's built-in functions are **available** by **default**; **no imports** are needed.*

Types of Functions

User-defined Functions

C++



```
#include <iostream>
using namespace std;
int add(int a, int b) {
    return a + b;
}
int main() {
    int num1, num2;
    cin >> num1 >> num2;
    cout << "Sum: " << add(num1, num2) << endl;
    return 0;
}
```

Python



```
def add(a, b):
    return a + b

num1 = int(input())
num2 = int(input())

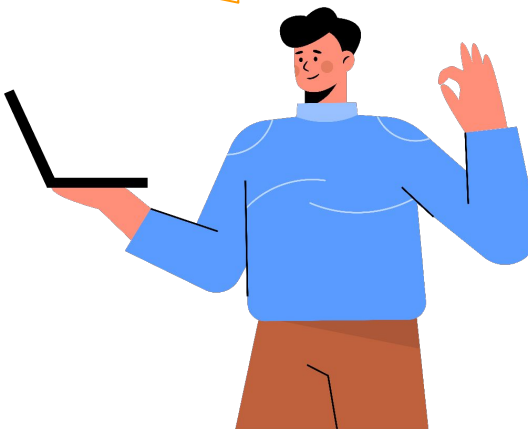
print("Sum: ", add(num1, num2))
```

Functions created by the programmer.

Why do we use functions?



We use functions to *break down* a program into *smaller, reusable parts*, making it *easier* to understand, maintain, and *avoid repeating code*.



Quiz Time!



Passing Parameters to Functions

Pass by Value

Pass by value is a parameter passing method where a **copy** of the **actual argument** is **passed to** the **function**. Changes made inside the function do **not affect** the **original variable**.

Passing Parameters to Functions by Value

Example

C++ 

```
void passByvalue(int a)
{
    a = a + 2;
    cout<<"Function: a = "<< a << endl;
}

int main(){
    int a = 2;
    passByvalue(a);
    cout<<"Main: a = "<< a << endl;
    return 0;
}
```

Output 

Function: a = 4

Main: a = 2

Passing Parameters to Functions

Pass by Reference

Pass by reference is a parameter-passing method where a function receives a **reference to** the **actual argument**, allowing **modifications inside** the **function** to **affect** the **original variable**.

Passing Parameters to Functions by Reference

Example

C++ 

```
void passByReference(int &a)
{
    a = a + 2;
    cout<<"Function: a = "<< a << endl;
}

int main(){
    int a = 2;
    passByReference(a);
    cout<<"Main: a = "<< a << endl;
    return 0;
}
```

Output 

Function: a = 4

Main: a = 4

Passing Array to Function

*In C++, when passing an array to a function, you only provide the **array's name**, which decays into a **pointer to its first element**, effectively making it pass by reference.*

Syntax </>

```
return_type function_name(array_type array_name[]){}  
//passing array to function
```

Passing Array to Function

Example

C++ 

```
void passingArray(int arr[], int len){  
    arr[1] = arr[3];  
    arr[0] = arr[4];  
}  
  
int main(){  
    int arr[] = {10,20,30,40,50};  
    passingArray(arr, 5);  
    for(int i = 0; i < 5; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

*This will update the main array
as it is always passed by
reference.*

Passing Array to Function

Example

C++ 

```
void passingArray(int arr[], int len){  
    arr[1] = arr[3];  
    arr[0] = arr[4];  
}  
  
int main(){  
    int arr[] = {10,20,30,40,50};  
    passingArray(arr, 5);  
    for(int i = 0; i < 5; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

Output 

50 40 30 40 50

Comparison of Functions: C++ vs Python

Feature	C++ Syntax	Python Syntax
<i>Function Declaration</i>	<code>int add(int, int);</code>	<i>No separate declaration available.</i>
<i>Function Definition</i>	<code>int add(int a, int b) { return a + b; }</code>	<code>def add(a, b): return a + b</code>
<i>Function Calling</i>	<code>sum = add(5, 10);</code>	<code>sum = add(5, 10)</code>
<i>Pass by Value</i>	<i>Default</i>	<i>Default</i>
<i>Pass by Reference</i>	<i>Use & (e.g., void modify(int &x))</i>	<i>Use mutable objects (lists, dicts) or global keyword</i>

Quiz Time!



Key Takeaways

✓ **Functions**

Reusable code blocks that take inputs, process them, and return outputs.

✓ **Parameter Passing:**

Pass by Value: Changes inside the function don't affect the original variable.

Pass by Reference: Changes inside the function affect the original variable.





THANK YOU





ALL THE BEST



Practice Problems

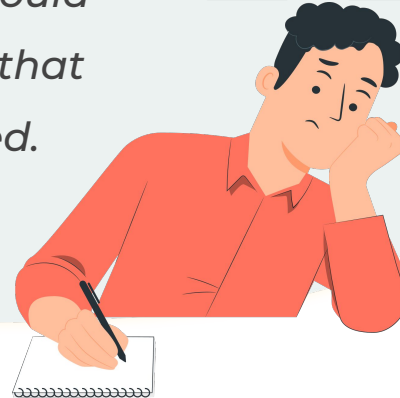
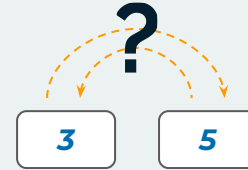


v/s



Swap Two Numbers

Write a program to swap two numbers using a function. The function should use pass-by-reference to ensure that the original values are modified.



Swap Two Numbers

Code

C++ 

```
#include <iostream>
using namespace std;
void interchange(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
int main() {
    int x = 5, y = 10;
    interchange(x, y);
    cout << x << " " << y;
}
```

Output 

10 5

Python 

```
def interchange(a, b):
    return b, a
x, y = 5, 10
x, y = interchange(x, y)
print(x, y)
```

Output 

10 5

Find Maximum

*Write a function, findMax, that takes three integers as input and returns the largest among them. Implement this function and call it within the **main()** function.*



Find Maximum

Code

C++ 

```
#include<bits/stdc++.h>
using namespace std;
int findMax(int a, int b, int c) {
    if (a >= b && a >= c) {
        return a;
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}
int main() {
    int x, y, z;
    cin >> x >> y >> z;
    int max = findMax(x, y, z);
    cout << max << endl;
    return 0;
}
```

Python 

```
def findMax(a, b, c):
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c

if __name__ == "__main__":
    x, y, z = map(int, input().split())
    max_value = findMax(x, y, z)
    print(max_value)
```

Sum of Digits



*Write a function `SumOfDigits` that takes an integer `n` as a parameter and returns the sum of the digits of the given number. Implement this function and use it in the **`main()` function.***

Sum of Digits

Code

C++ 

```
#include<bits/stdc++.h>
using namespace std;
int SumOfDigits(int n) {
    int sum = 0;
    while (n != 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
int main() {
    int num;
    cin >> num;
    int result = SumOfDigits(num);
    cout << result << endl;
    return 0;
}
```

Python 

```
def SumOfDigits(n):
    sum = 0
    while n != 0:
        sum += n % 10
        n //= 10
    return sum

if __name__ == "__main__":
    num = int(input())
    result = SumOfDigits(num)
    print(result)
```

Calculator



*Design a simple calculator that performs **Addition, Subtraction, Multiplication, and Division operations**. Implement the functionality using separate functions for each operation. The calculator should take two numbers and an operation as input and return the result.*

Calculator Code

C++



```
#include<bits/stdc++.h>
using namespace std;
double add(double a, double b) {
    return a + b;
}
double subtract(double a, double b) {
    return a - b;
}
double multiply(double a, double b) {
    return a * b;
}
double divide(double a, double b) {
    if (b != 0) {
        return a / b;
    } else {
        cout << "Error! Division by zero." << endl;
        return 0;
    }
}
```

Code continues...

```
int main() {
    double num1, num2;
    char op;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter an operator (+, -, *, /): ";
    cin >> op;
    cout << "Enter second number: ";
    cin >> num2;
    double result;
    if (op == '+') {
        result = add(num1, num2);
    } else if (op == '-') {
        result = subtract(num1, num2);
    } else if (op == '*') {
        result = multiply(num1, num2);
    } else if (op == '/') {
        result = divide(num1, num2);
    } else {
        cout << "Invalid operator!" << endl;
        return 1;
    }
    cout << "The result is: " << result << endl;
    return 0;
}
```

Python </>

```
def add(a, b):  
    return a + b  
def subtract(a, b):  
    return a - b  
def multiply(a, b):  
    return a * b  
def divide(a, b):  
    if b != 0:  
        return a / b  
    else:  
        return "Error! Division by zero."  
def main():  
    num1 = float(input("Enter first number: "))  
    op = input("Enter an operator (+, -, *, /): ")  
    num2 = float(input("Enter second number: "))
```

Code continues... 

```
    if op == "+":  
        result = add(num1, num2)  
    elif op == "-":  
        result = subtract(num1, num2)  
    elif op == "*":  
        result = multiply(num1, num2)  
    elif op == "/":  
        result = divide(num1, num2)  
    else:  
        result = "Invalid operator!"  
    print(f"The result is: {result}")  
if __name__ == "__main__":  
    main()
```